

Damindu Abeygunasekara

A complete technical breakdown of architecture, components, interactions, and design decisions behind the portfolio.

URL	Portfolio Website — Next.js / React
Author	Damindu Abeygunasekara
University	University of Kelaniya — BSc Computer Science
Email	daminduprasadith05@gmail.com
GitHub	github.com/daminduAb
Medium	medium.com/@adaminduprasadith
Date	June 12, 2026
Version	1.0

SECTION —

Table of Contents

1. Project Overview	3
Purpose · Tech Stack · Stack Diagram	
2. File & Folder Architecture	4
Directory tree · Key files	
3. Boot Sequence — BootLoader	5
Phases · CONFIRM flow · Hacking animation	
4. Main Page (page.tsx)	6
Profile mode · Projects mode · State management	
5. 3D Room — ThreeWorld.tsx	8
Scene graph · Components · Lighting · Interactivity	
6. UI Components	10
Navbar · ThemeToggle · PortfolioChatbot · ExperienceItem	
7. Supplementary Components	12
GithubGraph · TechStack · NeuralNetworkSim	
8. Blog / Writings Section	13
Medium integration · Article list	
9. Styling & Animation	14
Tailwind · Framer Motion · Dark mode	
10. Performance & Deployment	15
Next.js optimisations · Build · Vercel	

SECTION 1

Project Overview

Damindu Abeygunasekara's portfolio is a **single-page Next.js 14 application** that combines a cinematic 3D room, an interactive terminal boot sequence, an AI chatbot, and a macOS-inspired profile layout into one cohesive experience.

Goals

- Showcase projects, skills, and writings in a memorable, interactive way.
- Provide a professional, scannable CV-style profile.
- Demonstrate frontend engineering depth (Three.js, Framer Motion, Web Audio API, custom hooks).

Core Tech Stack

Framework	Next.js 14 (App Router)
Language	TypeScript
Styling	Tailwind CSS v3
Animation	Framer Motion
3D Engine	Three.js via React Three Fiber + Drei
Chatbot	Custom AI assistant (PortfolioChatbot)
Theme	next-themes (dark / light)
QR Code	qrcode.react
Icons	lucide-react · react-icons
Hosting	Vercel (recommended)

High-Level Data Flow

The application runs entirely client-side after the initial server render. On first load the BootLoader intercepts the viewport. Once the user types CONFIRM, the main page is revealed. All subsequent navigation (Profile ↔ Projects) is handled by React state — no page reloads.

SECTION 2

File & Folder Architecture

The project follows Next.js 14 App Router conventions. All components live in `app/components/` and are imported directly into `page.tsx`.

<code>app/</code>	Next.js App Router root
<code>page.tsx</code>	Main entry — all profile & project UI
<code>layout.tsx</code>	Root layout — font, meta, providers
<code>providers.tsx</code>	ThemeProvider wrapper (next-themes)
<code>components/</code>	All reusable UI components
<code>BootLoader.tsx</code>	Interactive terminal preloader
<code>ThreeWorld.tsx</code>	Full 3D room scene (Three.js / R3F)
<code>PortfolioChatbot.tsx</code>	Floating AI chat assistant
<code>ThemeToggle.tsx</code>	Sun/Moon icon button in navbar
<code>ExperienceItem.tsx</code>	Collapsible experience card
<code>GithubGraph.tsx</code>	GitHub contribution heatmap
<code>TechStack.tsx</code>	Animated tech logo grid
<code>NeuralNetworkSim.tsx</code>	Animated neural network canvas
<code>public/</code>	Static assets (images, audio)
<code>my.png</code>	Profile photo
<code>casua*.jpg</code>	Gallery images (12 photos)
<code>trackk.mp3</code>	Lofi background audio
<code>*.mp4 / *.png</code>	Project media

Key Architectural Decisions

- **App Router** — server components at the layout level; `page.tsx` is a client component ("use client").
- **Single page, dual mode** — Profile and Projects live in the same component, toggled by a React state variable.
- **No external state library** — `useState` + `useRef` + `useEffect` cover all state needs.
- **No database** — All content is hardcoded in TypeScript data arrays (projects, articles).

SECTION 3

Boot Sequence — BootLoader.tsx

The BootLoader is a fullscreen fixed overlay (z-index 200) rendered before the portfolio when `booted===false`. It creates an immersive hacker-terminal first impression.

Phase 1 — Boot lines

8 pre-defined strings are appended to state one by one on a 260 ms interval, simulating a Linux kernel boot. Each line with `[OK]` renders in green.

Phase 2 — CONFIRM prompt

After the last boot line, a 600 ms delay fires, then phase switches to 'prompt'. A yellow ASCII border box appears asking the user to type CONFIRM. An receives focus automatically.

Phase 3 — Hacking animation

On form submit the input is compared (case-insensitive) to 'CONFIRM'. Wrong input: the form shakes (x-axis keyframes) and shows a red bash error. Correct input: phase switches to 'hacking'. 15 lines of simulated shell commands (`sudo access`, `AES decrypt`, `./inject`) are appended at 110 ms intervals.

Phase 4 — Exit

After the last hack line, a 600 ms pause triggers `setExiting(true)`, which animates the container to `opacity:0 + scale:1.04` over 700 ms, then calls `onFinish()` — setting `booted=true` in `page.tsx`, which removes the BootLoader and reveals the portfolio.

Visual Design Elements

- CSS scanline overlay — repeating-linear-gradient every 4px creates CRT effect.
- ASCII logo — large DAMINDU text rendered in a block, hidden on mobile.
- Monospace font — entire component uses font-mono (Courier New).
- Color coding — green = OK lines, yellow = prompt border, white = GRANTED, red = error.
- Auto-scroll — `useRef + scrollIntoView` keeps the latest line visible.

State Variables

phase	'booting' 'prompt' 'hacking' 'done'
bootLines	string[] — lines shown during boot phase
hackLines	string[] — lines shown during hacking phase
input	string — controlled input value
shake	boolean — triggers x-axis shake animation on wrong input
exiting	boolean — triggers fade-out animation before <code>onFinish()</code>

SECTION 4

Main Page — page.tsx

page.tsx is the central client component (~1 040 lines). It manages all global state, renders both views, and hosts the navbar, modals, and chatbot.

State Management

booted	boolean — false until BootLoader calls onFinish()
mode	'profile' 'projects' — which view is active
time	string — IST clock updated every 1 s via setInterval
showQR	boolean — QR code modal visibility
showEasterEgg	boolean — starfield + blue aura overlay toggle
selectedProject	Project null — opens project detail modal
hoveredProject	number null — card hover highlight
isLofiPlaying	boolean — lofi audio play state
lofiVolume	number 0–1 — audio volume from range slider

Profile Mode

The profile view is a 680px-wide centered column styled after macOS app aesthetics — minimal color, generous whitespace, hairline dividers.

Sections (top to bottom):

- 1. Header — 72px round avatar + name + role + status dot + IST clock + lofi player
- 2. Bio — two short paragraphs about background and goals
- 3. NeuralNetworkSim — animated canvas neural network
- 4. Experience — three ExperienceItem collapsible entries
- 5. In Between — bordered card with personal interests
- 6. Education — single row (University of Kelaniya)
- 7. GitHub Contributions — GithubGraph heatmap
- 8. Tech Stack — TechStack animated logo grid
- 9. Writings — 9 real Medium articles as list rows (date · title · tag)
- 10. Gallery — infinite-scroll strip of 12 grayscale photos
- 11. Contact — five social/email links as list rows

Projects Mode

A CSS grid (1–3 columns responsive) of project cards. Each card shows: category badge, thumbnail area, title, short description, tech icons, stats grid, and GitHub/Demo links. Clicking opens a full modal with a video/image hero.

Projects: Blockchain Voting System · E-Commerce Platform · Eco Green Platform · WhatsApp AI Shop Agent

Easter Egg

Clicking the profile avatar toggles `showEasterEgg`. When active: a blue inset shadow covers the viewport edges, and 50 twinkling star dots (randomised position, duration, delay) are rendered as fixed absolutely-positioned divs.

Lofi Player

An `HTMLAudioElement` is created lazily on first play (stored in a `useRef` to survive re-renders). It loops `/public/trackk.mp3`. Volume is controlled by a range input that slides in/out with an `AnimatePresence` transition.

SECTION 5

3D Room — ThreeWorld.tsx

ThreeWorld.tsx renders a fully interactive 3D room using **React Three Fiber** (R3F) — a React renderer for Three.js. The Canvas component from R3F replaces the standard Three.js setup/render loop boilerplate.

Canvas Setup

camera	position [0, 2.5, 9], fov 50
shadows	enabled — shadow-mapSize 2048×2048 on spotLight
fog	linear fog #010b18, near 10, far 26
background	color #010b18 (near-black)

Lighting

- ambientLight — 0.3 intensity, blue tint (#1e40af)
- hemisphereLight — sky #60a5fa / ground #010b18
- spotLight — ceiling position [0,8,1], angle 0.38, penumbra 1, intensity 40, casts shadows
- 4 accent pointLights — purple/green/pink/blue at room corners
- Per-object pointLights on NeonPillars and FloatingOrbs
- Desk lamp warm pointLight (#fef3c7)

Scene Components

RoomShell	14×7×14 unit box (BackSide) = walls/ceiling/floor. Ceiling light panel, 5 neon wall strips.
NeonPillars	4 corner pillars, each a different accent color (cyan/purple/green/pink) with a sphere cap and a pointLight.
Platform	Cylinder under the desk with a glowing torus ring edge.
Rug	Flat box under the desk area with faint blue emissive glow.
Chair	Box seat + backrest + 4 cylinder legs + neon strip on backrest.
Desk	Wide box surface with 4 leg meshes and a front edge glow strip.
MainMonitor	Bezel box + screen plane + Html overlay (ScreenContent) + stand neck/base. pointLight pulses the screen color.
SideMonitor	Smaller rotated monitor showing a simulated terminal with git status output via Html.
Plant	Pot cylinder + 5 sphere leaves, slowly sways via useFrame.
MousePad	Box pad + mouse box + thin emissive scroll-wheel line.
Keyboard	Box body + 3 translucent key-row planes.
Bookshelf	Frame box + 4 shelves + up to 12 colored book meshes.

WallArtPanel	Thin emissive box on wall, pulses emissiveIntensity in useFrame.
FloatingOrbs	4 orbs (sphere + torus ring + pointLight), each bobbing on a sin wave via useFrame.
HologramPanel	RoundedBox + Float + Html button — 3 panels (About/Skills/Projects). Clicking sets activePanel state → changes ScreenContent.

Interactivity

Three HologramPanel components each have an onClick handler that calls setActivePanel(panelKey). The activePanel state is passed down to the Computer component → MainMonitor → ScreenContent, which renders one of three HTML panels (About / Skills / Projects) inside an R3F overlay.

Drei Helpers Used

ContactShadows	Float	Grid	Html	OrbitControls	RoundedBox
Sparkles	Text				

OrbitControls

autoRotate	true — speed 0.3
minDistance	5 / maxDistance: 10
minPolarAngle	PI/3.8 (can't look below desk)
maxPolarAngle	PI/2.1 (can't go underground)
enablePan	false

SECTION 6

UI Components

Navbar

Fixed bottom-center floating pill — glass morphism style (backdrop-blur-xl, bg-white/80 dark:bg-zinc-900/85, subtle border).

Contains (left to right):

- Profile button — fills black when mode==='profile'
- Projects button — fills black when mode==='projects'
- Separator line
- Social links — GitHub, LinkedIn, X, Medium (SVG), Discord
- Separator line
- QR button — opens resume QR modal
- ThemeToggle — Sun/Moon icon

ThemeToggle

A 32×32 px icon button inside the navbar. Uses next-themes useTheme() hook. Renders Sun icon in dark mode, Moon in light mode. Skips render until mounted (avoids hydration mismatch).

PortfolioChatbot

Floating chat assistant positioned bottom-left (bottom-6 left-6). Built entirely from scratch — no external chat SDK.

Sub-components:

- TypewriterText — character-by-character typing with a blinking cursor (18 ms interval)
- ToneRipple — 3 concentric animated rings that pulse on open
- ThinkingDots — 3 bouncing dots shown while waiting for reply

State:

isOpen	boolean — chat window visible
isMinimized	boolean — collapsed to header bar only
messages	Message[] — chat history
hasUnread	boolean — notification dot on button
ripple	boolean — triggers ToneRipple on open

Audio:

Uses Web Audio API (no audio files). playSendSound() creates a short ascending sine wave (440→880 Hz, 180 ms). playReceiveSound() creates two soft pings at 880 and 1046 Hz.

Chat Window:

Max-height animates between 64px (minimized) and 400px (open) via CSS transition. Frosted glass background (backdrop-blur-xl). Message bubbles: user = black/white, assistant = gray.

ExperienceItem

Collapsible section used for Experience entries. Props: title, role, collapsible (boolean), link (optional URL). When collapsible=true, an expand/collapse arrow toggles a children div with a smooth height animation via Framer Motion AnimatePresence.

SECTION 7

Supplementary Components

GithubGraph

Fetches GitHub contribution data for user daminduAb and renders a heatmap calendar grid (similar to GitHub's contribution chart). Uses the react-activity-calendar library.

TechStack

An animated grid of technology logos. Each logo is rendered as an SVG icon from react-icons. Logos include: React, Next.js, TypeScript, Tailwind CSS, Three.js, Python, FastAPI, MongoDB, Solidity, Node.js, Supabase, MySQL, Stripe, OpenAI, and more.

Hover interaction: individual logos scale up or glow on hover using Framer Motion whileHover.

NeuralNetworkSim

An HTML5 Canvas animation that draws an animated neural network topology — nodes connected by weighted edges. Each frame uses requestAnimationFrame to update node pulse states and edge opacity, creating a living network visualization. Rendered between the bio section and experience on the profile page.

SECTION 8

Blog / Writings Section

The Writings section fetches no data at runtime — all 9 articles are hardcoded as a TypeScript array. Each entry links directly to the Medium article URL. The section uses a macOS list-row style: date left · title center · tag right, separated by hairline dividers.

Medium Profile:

medium.com/@adaminduprasadith

Article List

Date	Tag	Title
Jun 2025	AI / LLM	Fine-tuning vs RAG: Stop Guessing, Start Choosing Wisely
Jun 2025	AI / LLM	Attention Is All You Need — But Do You Actually Understand It?
May 2025	Python	Learn SOLID Principles in 2 Hours — Complete Beginner Guide
May 2025	ML	What even is a machine learning model?
Apr 2025	Web Dev	Build Full-Stack Web Apps with the MERN Stack
Apr 2025	DevOps	Monorepo in GitHub
Apr 2025	DevOps	Jenkins Made Simple: A Beginner-Friendly Guide for Developers
Apr 2025	Python	FastAPI: Build Lightning-Fast APIs with Minimal Code
Apr 2025	Architecture	Microservices Made Simple: A Beginner-Friendly Guide

SECTION 9

Styling & Animation

Tailwind CSS

All styles are utility-first Tailwind classes. No custom CSS files except globals.css (which defines the animate-infinite-scroll keyframe for the gallery strip).

Dark mode uses the class strategy — a dark class on toggles all dark: prefixed utilities. next-themes manages this automatically.

Framer Motion

Used throughout for:

- Page transitions — AnimatePresence + motion.main with opacity/y enter/exit on mode switch
- Card hover — whileHover={{ y: -4 }} on project cards
- BootLoader shake — animate={{ x: [0,-8,8,-6,6,0] }} on wrong CONFIRM
- Chatbot window — spring damping 22/stiffness 200 open/close
- Easter egg stars — individual opacity/scale ping animations
- Lofi volume slider — width 0→48 AnimatePresence slide

Dark Mode System

Provider	ThemeProvider from next-themes wraps the app in providers.tsx
Strategy	class — adds dark class to
Toggle	ThemeToggle component in navbar — Sun ↔ Moon icon
Persistence	next-themes stores preference in localStorage automatically
SSR	useTheme returns undefined on server; mounted state guards the render

Infinite Gallery Scroll

The photo gallery uses a CSS animation defined in globals.css:

```
@keyframes infinite-scroll { from { transform: translateX(0); } to { transform: translateX(-50%); } }
```

The image list is duplicated (images + images) so the loop is seamless. A mask-image linear-gradient fades the edges.

SECTION 10

Performance & Deployment

Next.js Optimisations

- **next/image** — all images use the Image component for automatic WebP conversion, lazy loading, and size optimisation.
- **Dynamic imports** — ThreeWorld is conditionally rendered only when the 3D room section is visible; R3F uses Suspense fallback.
- **Font** — system fonts via Tailwind (font-sans, font-mono) — no external font download.
- **Client components** — only page.tsx and components that need browser APIs are marked 'use client'. Layout is a server component.

Build

npm run build — Next.js 14 static + SSR hybrid.

npm run dev — local dev server on localhost:3000.

Recommended Deployment — Vercel

1. Push to GitHub repository.
2. Import repository in Vercel dashboard.
3. Framework preset: Next.js (auto-detected).
4. No environment variables required.
5. Vercel auto-deploys on every git push to main.

Summary

Damindu's portfolio is a technically sophisticated single-page application that blends a cinematic 3D environment, a terminal preloader with user interaction, a macOS-inspired profile layout, an AI chatbot, and a complete writings section — all built with Next.js 14, TypeScript, Tailwind CSS, Framer Motion, and React Three Fiber.

Report generated on June 12, 2026 at 21:57 · Damindu Abeygunasekara · daminduprasadith05@gmail.com